



DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

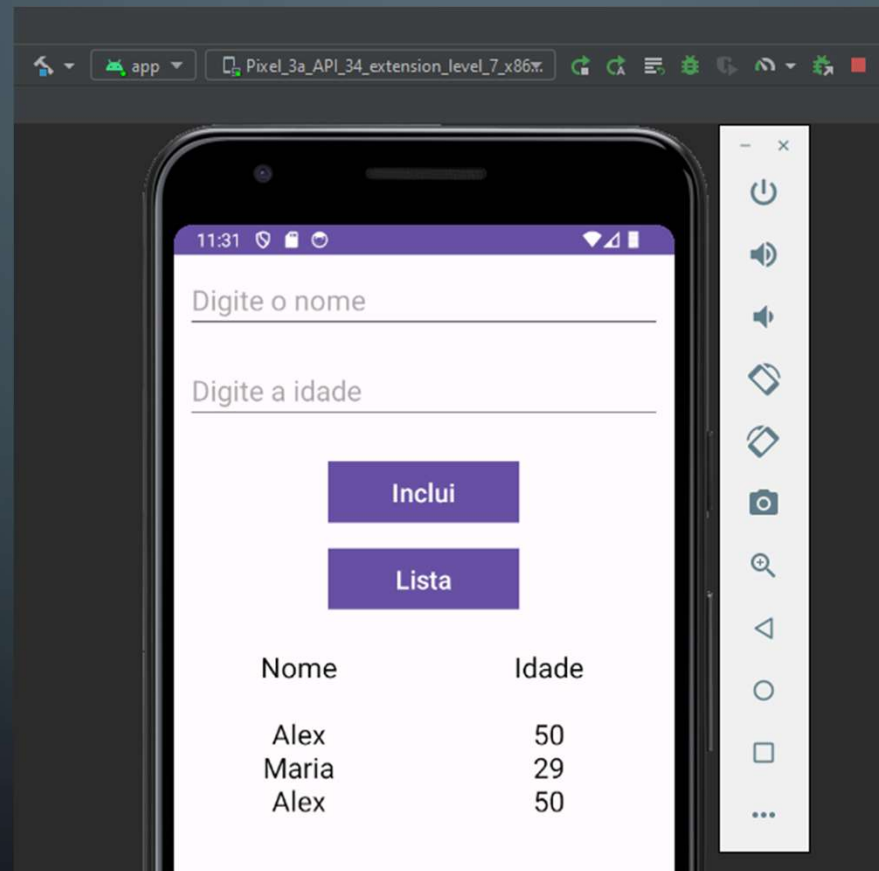
ALEX@EINSTEINLIMEIRA.COM.BR

SQLITE – ALTERANDO DADOS MANUALMENTE

- De dentro do Android Studio conseguimos gerar comandos SQL para alterar os dados diretamente no nosso banco SQLite.
- Podendo dessa forma checar o funcionamento do nosso app ou mesmo incluir, excluir ou alterar dados das tabelas para testar o aplicativo.
- Essa alteração se dá direto no nosso emulador, por isso o app precisa estar sendo executado.

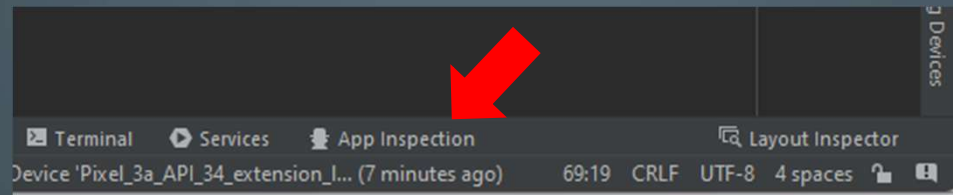
SQLITE – ALTERANDO DADOS MANUALMENTE

- Execute o programa da aula passada e espere o emulador abrir:

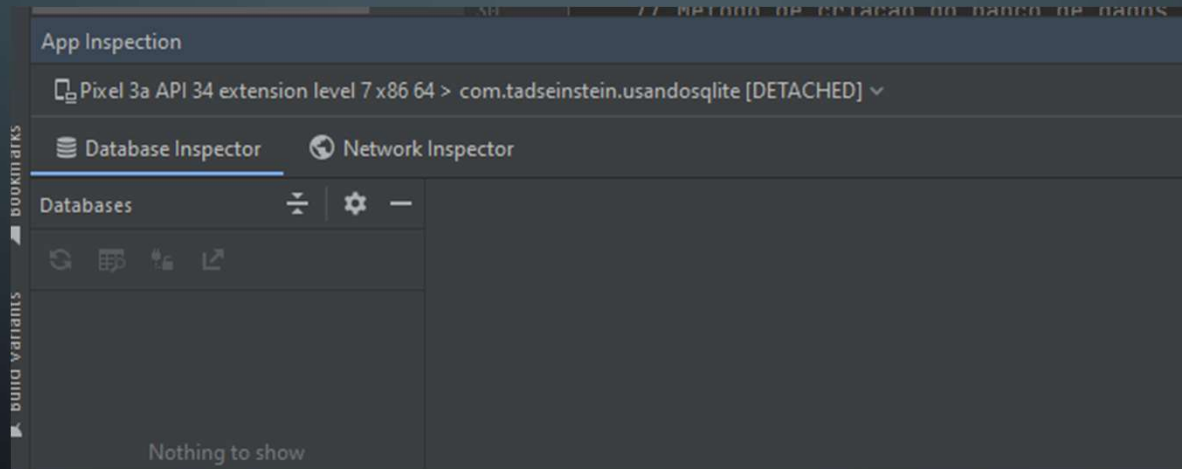


SQLITE – ALTERANDO DADOS MANUALMENTE

- No rodapé do Android Studio clique em **App Inspection**:

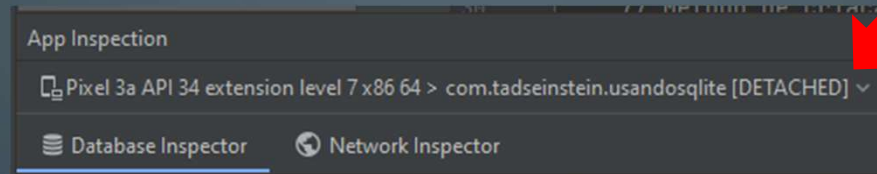


- Vai abrir essa janela na parte de baixo da tela:

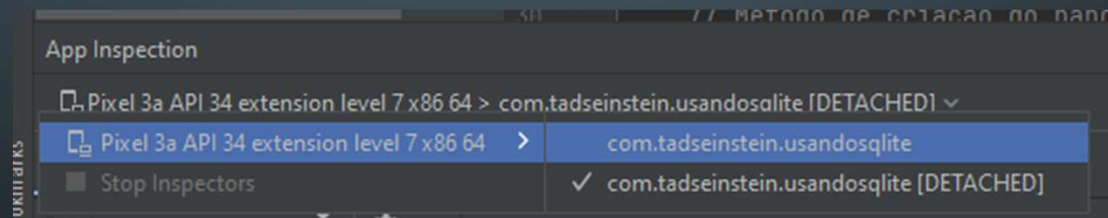


SQLITE – ALTERANDO DADOS MANUALMENTE

- Se no topo da janela estiver escrito **DETACHED**, precisamos fazer a conexão:

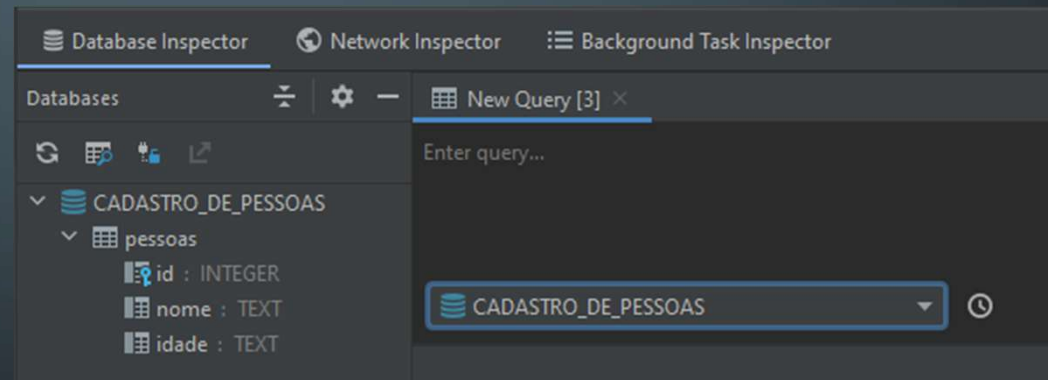
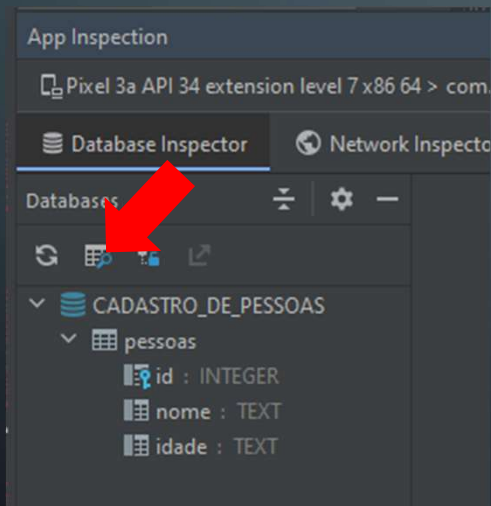


- Clique sobre ele, depois no nome do projeto para fazer a conexão:



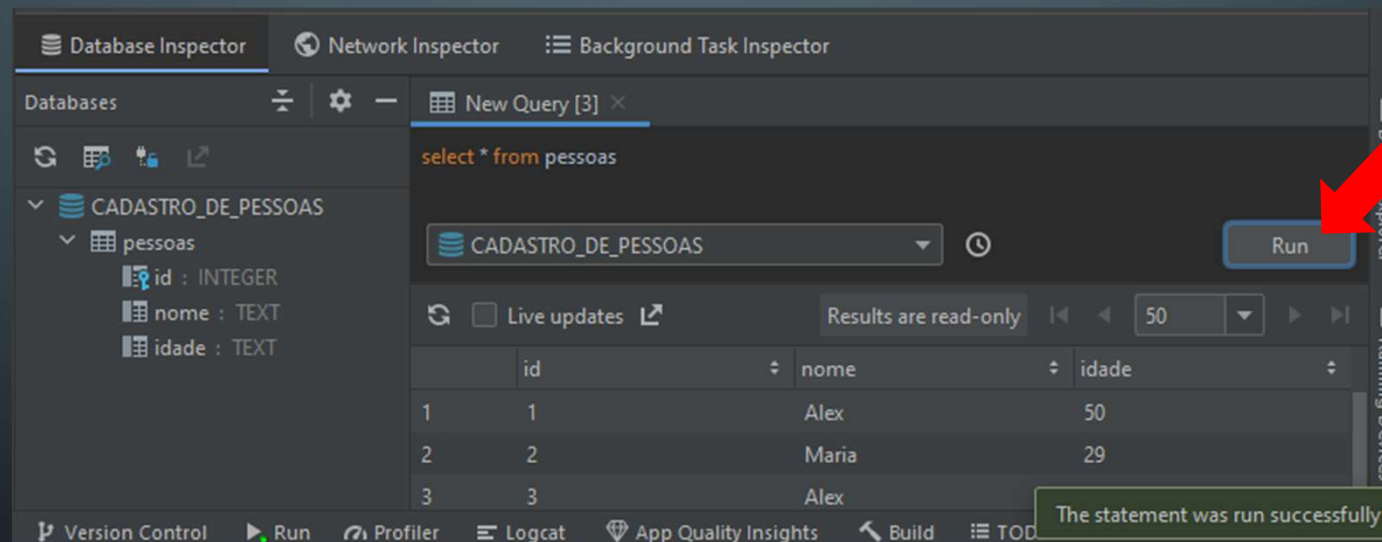
SQLITE – ALTERANDO DADOS MANUALMENTE

- Agora ele esta exibindo nosso banco de dados e suas tabelas.
- Clique no botão indicado abaixo para abrir uma caixa onde podemos fazer nossa Query:



SQLITE – ALTERANDO DADOS MANUALMENTE

- Pronto, nessa caixa podemos escrever nossas SQL para manutenção nos dados do banco, para executar clique em **Run**:



SQLITE

- Vamos alterar nosso app para incluir um botão para excluir e outro para alterar dados:

```
<Button
    android:id="@+id/btn_altera"
    android:layout_width="150dp"
    android:layout_gravity="center"
    android:background="#2196F3"
    android:text="Alterar"
    android:textColor="#ffffff"
    android:textSize="20sp"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"/>

<Button
    android:id="@+id/btn_exclui"
    android:layout_width="150dp"
    android:layout_gravity="center"
    android:background="#2196F3"
    android:text="Excluir"
    android:textColor="#ffffff"
    android:textSize="20sp"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"/>
```

Digite o nome

Digite a idade

Inclui

Lista

Alterar

Excluir

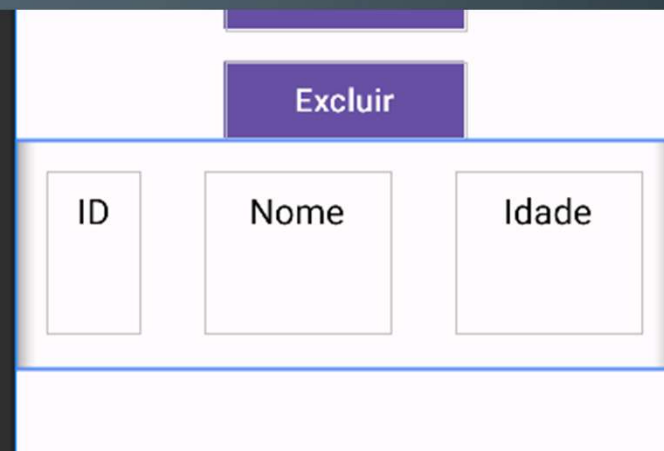
Nome

Idade

SQLITE

- Vamos utilizar o **ID** para fazer essas alterações, para isso vamos exibir o mesmo:

```
<TextView
    android:id="@+id/txt_id"
    android:textAlignment="center"
    android:layout_weight="1"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_margin="20dp"
    android:text="ID\n\n"
    android:textSize="22sp"
    android:padding="10dp"
    android:textColor="#000000"/>
```



SQLITE

- Para a tela ficar igual a do slide anterior, acerte o `layout_weight` e o `layout_width` dos componentes conforme abaixo:

```
<TextView
    android:id="@+id/txt_id"
    android:textAlignment="center"
    android:layout_weight="1"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_margin="20dp"
    android:text="ID\n\n"
    android:textSize="22sp"
    android:padding="10dp"
    android:textColor="#000000"/>
```

```
<TextView
    android:id="@+id/txt_nome"
    android:textAlignment="center"
    android:layout_weight="2"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_margin="20dp"
    android:text="Nome\n\n"
    android:textSize="22sp"
    android:padding="10dp"
    android:textColor="#000000"/>
```

```
<TextView
    android:id="@+id/txt_idade"
    android:textAlignment="center"
    android:layout_weight="2"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_margin="20dp"
    android:text="Idade\n\n"
    android:textSize="22sp"
    android:padding="10dp"
    android:textColor="#000000"/>
```

SQLITE

- Adicione a linha abaixo no método `botaoLista()`:

```
@SuppressWarnings("Range")
private fun botaoLista() {
    // Conecta o banco de dados na variavel db
    val db = DBHelper(context: this, factory: null)

    // Chamamos o metodo getName para pegar os registros
    // e associá-los ao cursor
    val cursor = db.getName()

    // Move o cursor para o primeiro registro
    cursor!!.moveToFirst()

    // Manda os registros para a tela
    do {
        binding.txtNome.append(cursor.getString(cursor.getColumnIndex(DBHelper.NAME_COL)) + "\n")
        binding.txtIdade.append(cursor.getString(cursor.getColumnIndex(DBHelper.AGE_COL)) + "\n")
        binding.txtId.append(cursor.getString(cursor.getColumnIndex(DBHelper.ID_COL)) + "\n")
    } while (cursor.moveToNext())

    // Fecha o cursor
    cursor.close()
}
```

SQLITE

- Vamos fazer mais uma alteração no método `botaoLista()` para limpar a lista toda vez que ele é clicado:

```
@SuppressWarnings("Range")
private fun botaoLista() {
    //Limpa a lista
    binding.txtId.text = "ID\n\n";
    binding.txtNome.text = "Nome\n\n";
    binding.txtIdade.text = "Idade\n\n";

    // Conecta o banco de dados na variavel db
    val db = DBHelper(context, this, factory, null)
```

- Agora vamos dar mais uma melhoria no visual e permitir que o ID seja informado para exclusão ou manutenção.

SQLITE

- Insira mais um **LinearLayout**: e acerte as propriedades dos dois botões:

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <Button
        android:id="@+id/btn_inclui"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_marginStart="30dp"
        android:layout_marginEnd="15dp"
        android:background="#2196F3"
        android:text="Inclui"
        android:textColor="#ffffff"
        android:textSize="20sp"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"/>
```

```
<Button
    android:id="@+id/btn_lista"
    android:layout_width="0dp"
    android:layout_weight="1"
    android:layout_marginStart="15dp"
    android:layout_marginEnd="30dp"
    android:background="#2196F3"
    android:text="Lista"
    android:textColor="#ffffff"
    android:textSize="20sp"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"/>
```

```
</LinearLayout>
```

Digite o nome

Digite a idade

Inclui Lista

Alterar

Excluir

ID Nome Idade

SQLITE

- Agora com os outros dois botões, inclua um `EditText` para o `ID`:

```
<EditText
    android:id="@+id/edit_id"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="30dp"
    android:layout_marginStart="30dp"
    android:layout_marginEnd="30dp"
    android:inputType="number"

    android:textSize="22sp"
    android:hint="Informe o ID"/>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <Button
        android:id="@+id/btn_altera"
        android:layout_width="0dp"
        android:layout_weight="1"
```

```
        android:layout_marginStart="30dp"
        android:layout_marginEnd="15dp"
        android:background="#2196F3"
        android:text="Alterar"
        android:textColor="#ffffff"
        android:textSize="20sp"
        android:layout_height="wrap_content"/>
```

```
    <Button
        android:id="@+id/btn_exclui"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_marginStart="15dp"
        android:layout_marginEnd="30dp"
        android:background="#2196F3"
        android:text="Excluir"
        android:textColor="#ffffff"
        android:textSize="20sp"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

Digite o nome

Digite a idade

Inclui Lista

Informe o ID

Alterar Excluir

ID Nome Idade

SQLITE

- Vamos criar o método para alteração de registros dentro do **DBHelper.kt**:

```
// Método para modificar registros no banco de dados
fun updateRec(id: Int, name : String, age : String ){
    // Cria variável do tipo ContentValues
    val values = ContentValues()

    // Insere os dados na variável criada
    // utilizando a constante com o nome do campo
    // criada no início da classe
    values.put(ID_COL, id)
    values.put(NAME_COL, name)
    values.put(AGE_COL, age)

    // Cria a variável db como sendo
    // nosso banco de dados com direitos de gravação
    val db = this.writableDatabase

    // modifica o registro no banco de dados
    db.update(TABLE_NAME, values, whereClause: "id=?", arrayOf(id.toString()))

    // Fecha o banco de dados
    db.close()
}
```

SQLITE

```
// modifica o registro no banco de dados  
db.update(TABLE_NAME, values, whereClause: "id=?", arrayOf(id.toString()))
```

- Parâmetros do **update**:
 - Nome da tabela;
 - Valores dos campos, variável do tipo **ContentValues**;
 - Clausula **where** do SQL, a **?** Será substituída pelos valores do próximo parâmetro;
 - Valores para a clausula **where**, em formato de **array** contendo **strings**

SQLITE

- Para incluir o código do botão **Alterar**, precisamos primeiro permitir seu acesso, dentro da **MainActivity.kt** faça as alterações:

```
binding.btnInclui.setOnClickListener(this)
binding.btnLista.setOnClickListener(this)
binding.btnAlterar.setOnClickListener(this)
}

override fun onClick(v: View) {
    if (v.id == R.id.btn_inclui) {
        botaoInclui()
    }
    else if (v.id == R.id.btn_lista) {
        botaoLista()
    }
    else if (v.id == R.id.btn_altera) {
        botaoAlterar()
    }
}
```

SQLITE

- Código do botão **Alterar**:

```
private fun botaoAlterar() {  
    // Conecta o banco de dados na variavel db  
    val db = DBHelper( context: this, factory: null)  
  
    // carrega os dados  
    val id = binding.editId.text.toString().toInt()  
    val name = binding.editNome.text.toString()  
    val age = binding.editIdade.text.toString()  
  
    // Altera no banco de dados  
    db.updateRec( id, name, age)  
  
    Toast.makeText( context: this, text: name + " alterado no banco!", Toast.LENGTH_LONG).show()  
  
    // Limpa os EditText  
    binding.editId.text.clear()  
    binding.editNome.text.clear()  
    binding.editIdade.text.clear()  
}
```

SQLITE

- Da forma que fizemos a alteração acontece, mas ele não pede uma confirmação antes de efetivar a mesma.
- Podemos utilizar um `AlertDialog` para confirmar uma operação, mas esse componente não para a execução do programa para esperar o usuário escolher a opção desejada.
- Por isso temos que mudar um pouco a forma de escrever o programa.
- Vamos modificar o nosso `botão Altera` para exibir uma caixa de confirmação:

SQLITE

- Primeiro crie uma nova função e mova todo o código do `botaoAlterar` para ela:

```
private fun botaoAlterar() {  
    }  
  
private fun alterar() {  
    // Conecta o banco de dados na variável db  
    val db = DBHelper( context: this, factory: null)  
  
    // carrega os dados  
    val id = binding.editId.text.toString().toInt()  
    val name = binding.editNome.text.toString()  
    val age = binding.editIdade.text.toString()  
  
    // Altera no banco de dados  
    db.updateRec(id, name, age)  
  
    Toast.makeText( context: this, text: name + " alterado no banco!", Toast.LENGTH_LONG).show()  
  
    // Limpa os EditText  
    binding.editId.text.clear()  
    binding.editNome.text.clear()  
    binding.editIdade.text.clear()  
}
```

SQLITE

- Agora vamos escrever o código do botão Altera:

```
private fun botaoAlterar(){  
    AlertDialog.Builder( context: this) AlertDialog.Builder  
        .setMessage( "Confirma a alteração do registro" ) AlertDialog.Builder!  
        .setPositiveButton( text: "Sim", DialogInterface.OnClickListener { dialogInterface, which -> altera() })  
        .setNegativeButton( text: "Não", listener: null)  
        .create() AlertDialog  
        .show()  
}
```



- Note que no botão sim (`setPositiveButton`) ele chama o método `altera()` que criamos no slide anterior.
- Como não precisamos fazer nada no botão `não`, ele aponta para `null`.

SQLITE

- Antes de fazer o botão Excluir, precisamos criar o método de exclusão dentro da **DBHelper.kt**:

```
// Método para deletar registros no banco de dados
fun delete(id: Int ){

    // Cria a variavel db como sendo
    // nosso banco de dados com direitos de gravação
    val db = this.writableDatabase

    // Deleta do banco de dados
    db.delete(TABLE_NAME, whereClause: "id=?", arrayOf(id.toString()))

    // Fecha o banco de dados
    db.close()
}
```

- Só precisamos do ID que será excluído, os parâmetros do método **delete** são parecidos com os do método **update**, só não precisamos passar os dados que serão alterados.

SQLITE

- Vamos fazer o botão excluir. Primeiro associe ele a uma função na `MainActivity.kt`:

```
binding.btnAlterar.setOnClickListener(this)
binding.btnExcluir.setOnClickListener(this)
}

override fun onClick(v: View) {
    if (v.id == R.id.btn_incluir) {
        botaoIncluir()
    }
    else if (v.id == R.id.btn_lista) {
        botaoLista()
    }
    else if (v.id == R.id.btn_alterar) {
        botaoAlterar()
    }
    else if (v.id == R.id.btn_excluir) {
        botaoExcluir()
    }
}
```

SQLITE

- Agora vamos fazer o código dessa função, como vamos pedir uma confirmação usaremos o **AlertDialog**:

```
private fun botaoExclui() {  
    AlertDialog.Builder( context: this) AlertDialog.Builder!  
        .setMessage( "Confirma a exclusão do registro?" ) AlertDialog.Builder!  
        .setPositiveButton( text: "Sim", DialogInterface.OnClickListener { dialogInterface, which -> exclui() })  
        .setNegativeButton( text: "Não", listener: null)  
        .create() AlertDialog  
        .show()  
}
```

SQLITE

- O código de exclusão:

```
private fun exclui() {  
    // Conecta o banco de dados na variavel db  
    val db = DBHelper(context: this, factory: null)  
  
    // Carrega a variavel ID  
    val id = binding.editId.text.toString().toInt()  
  
    // deleta no banco de dados  
    db.delete(id)  
  
    Toast.makeText(context: this, text: binding.editNome.text.toString() + " excluido no banco!", Toast.LENGTH_LONG).show()  
  
    // Limpa o EditText  
    binding.editId.text.clear()  
}
```